

User Defined Data Structures: Queue and Priority Queue in Python



by
Dr M Rajasekhara Babu

School of
Computer Science and Engineering
<https://www.rajababu.org>

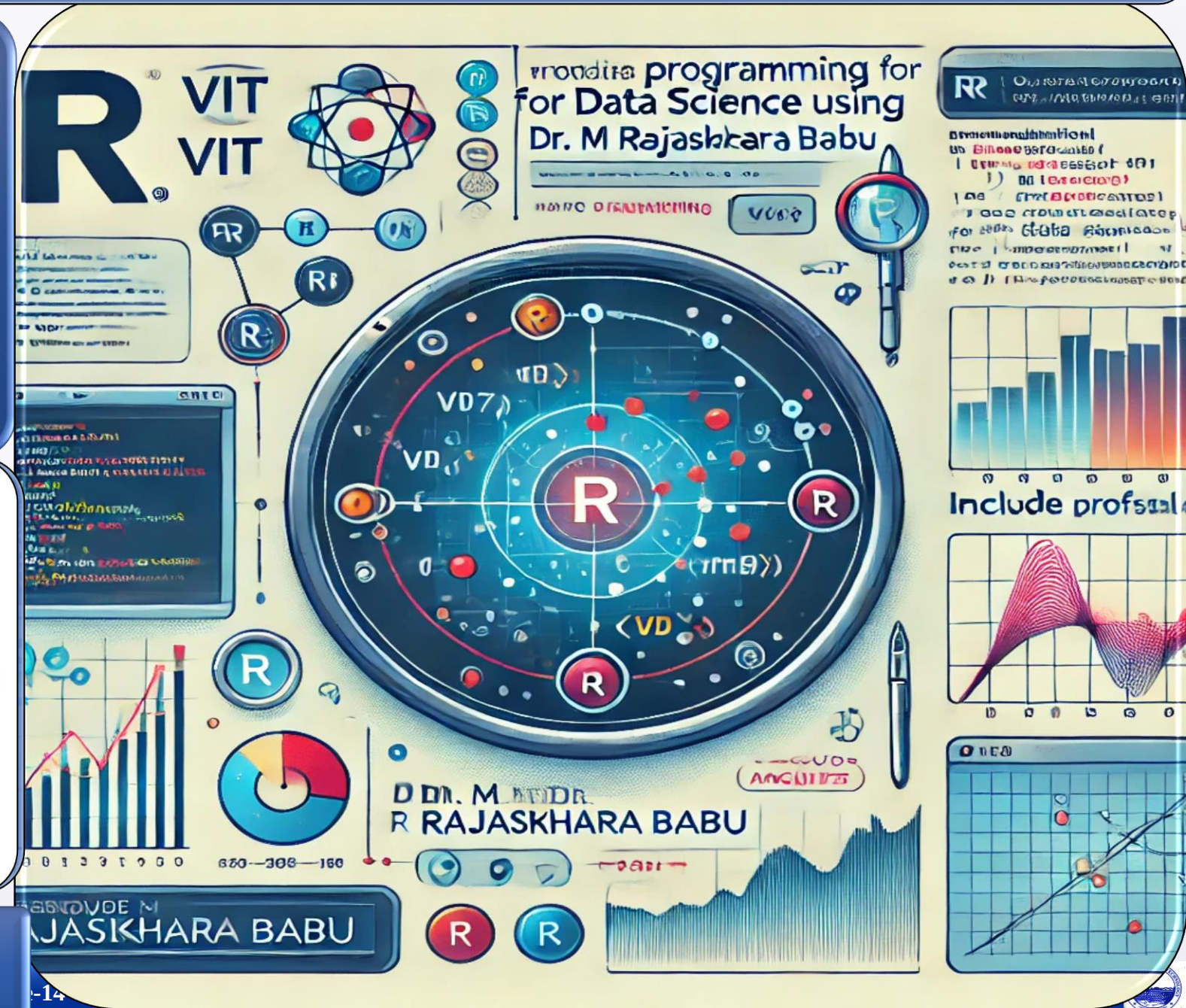


VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India



Objectives & Teaching Learning Material



To explain the concepts of Queue and Priority Queue, including FIFO ordering and priority-based processing principles



To demonstrate Queue and Priority Queue operations such as Enqueue, Dequeue, Front, Rear, and Display using Python implementations



To implement user-defined Queue and Priority Queue classes in Python and analyze their efficiency and real-world applications

Teaching + Learning

Material:

LCD, White board Marker, Presentation slides

Learning outcomes

Upon successful completion of this lecture, students will be able to:



Explain the structure, behavior, and operational principles of Queue and Priority Queue data structures



Implement Queue and Priority Queue operations using Python lists and object-oriented programming techniques



Analyze and apply Queue and Priority Queue data structures to solve scheduling, resource allocation, networking, and task-prioritization problems

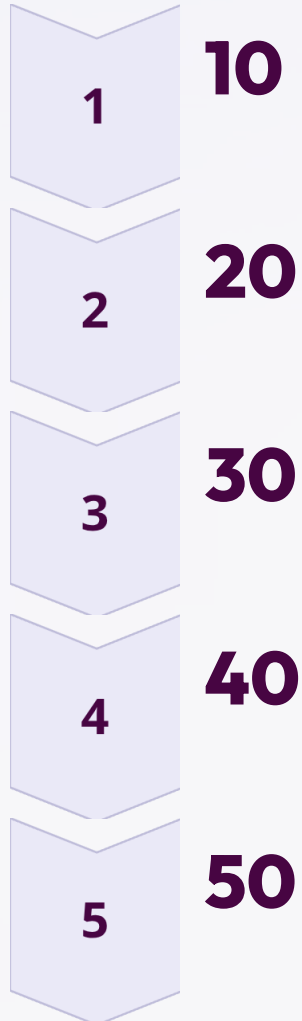


Session Plan

Time in minutes	Topic	Learning Aid & Methodology	Faculty Approach	Student Activity	Skill and Competency Developed
05	Re-Cap	Quiz Presentation	Questions Organizes	Answers Identifies	Knowledge Understanding
10	Introduction to Linear Data Structures and Queue Fundamentals	Presentation	Presentation	Listens	Remembering
10	Queue Operations and Python Implementation	Explains	Explains	Notes	Understanding
15	Priority Queue Concepts and Implementation	Case Study	Organizes	Observes	Applying
10	Comparison, Complexity Analysis, and Real-World Applications	Discussion	Facilitates	Answers	Analyzing



In a linear data structure, elements are arranged sequentially — each element is connected to the next in a single line.



Key examples of linear data structures covered in this lecture:

Queue

FIFO — First In, First Out ordering

Priority Queue

Elements processed by assigned priority value

What is a Queue?



A Queue is a linear data structure that follows the FIFO principle:

FIFO

First In, First Out — the first element inserted into the queue is the first element removed. Just like a real-world line, no one can cut ahead.

Real-Life Example: Ticket Counter

Consider four customers arriving at a ticket counter in order: $A \rightarrow B \rightarrow C \rightarrow D$. They are served in the exact order they arrived — this is FIFO behavior.

01

Customer A

Arrives first — served first

02

Customer B

Arrives second — served second

03

Customer C

Arrives third — served third

04

Customer D

Arrives fourth — served fourth

- ❏ No element can be removed before those that arrived earlier — this is the defining rule of a Queue.

Queue Operations

A Queue supports six fundamental operations that define how elements are inserted, removed, and inspected.

Operation	Description
Enqueue	Insert an element at the rear of the queue
Dequeue	Remove an element from the front of the queue
Front	View the first (front) element without removing it
Rear	View the last (rear) element without removing it
isEmpty	Check whether the queue is empty
Display	Show all current contents of the queue

Queue Structure

Insertion happens at the Rear.
Deletion happens at the Front.

Front

10 → 20 → 30 → 40

Rear

Python Implementation

```
queue = []
```

```
# Enqueue
```

```
queue.append(10)
```

```
queue.append(20)
```

```
queue.append(30)
```

```
print(queue)
```

```
# Output: [10, 20, 30]
```

Python lists make queue creation straightforward. Use `append()` to enqueue at the rear and `pop(0)` to dequeue from the front.

Deque Operation

```
queue = [10, 20, 30]
item = queue.pop(0)
print(item)
print(queue)
```

Output:

```
10
[20, 30]
```

`pop(0)` removes and returns the front element. Index `[0]` accesses the front and `[-1]` accesses the rear without removing them.

Front & Rear Access

```
queue = [10, 20, 30]
print("Front =", queue[0])
print("Rear =", queue[-1])
```

Output:

```
Front = 10
Rear = 30
```

User Defined Queue Class

A complete, user-defined Queue class encapsulates all operations cleanly using Python's object-oriented features.

```
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, data):
        self.items.append(data)

    def dequeue(self):
        if len(self.items) == 0:
            return "Queue Empty"
        return self.items.pop(0)

    def front(self):
        return self.items[0]

    def display(self):
        print(self.items)

q = Queue()
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
q.display()
print("Deleted =", q.dequeue())
q.display()
```

Output: [10, 20, 30] → Deleted = 10 → [20, 30]

Applications of Queue

Queues are fundamental to many real-world computing and service systems where order of arrival must be preserved.



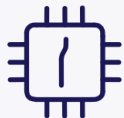
Ticket Reservation

Booking systems process requests in arrival order



Printer Queue

Print jobs are processed in the order submitted



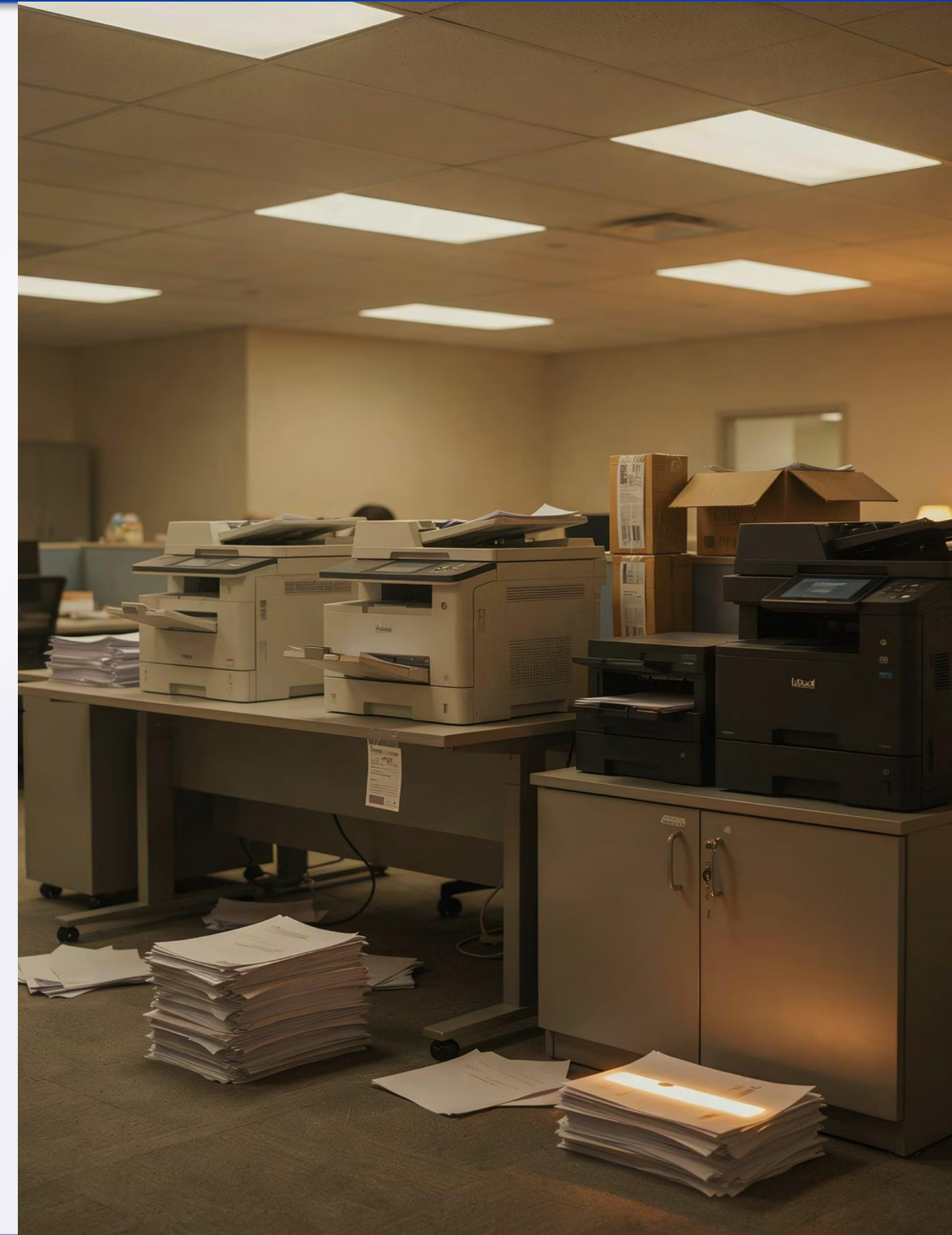
CPU Scheduling

Processes queued and executed sequentially



Network Transmission

Data packets transmitted in order received



What is a Priority Queue?

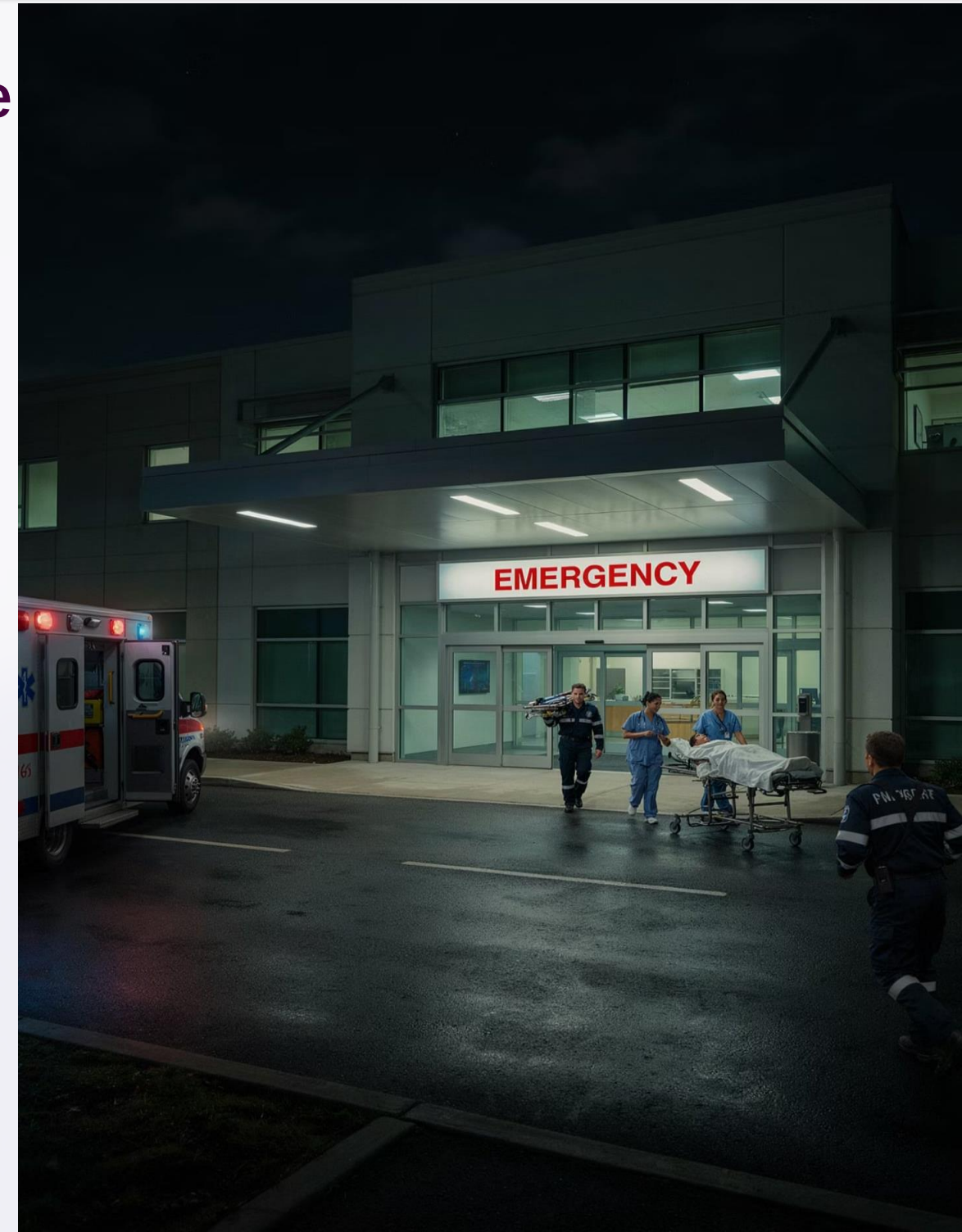
A Priority Queue is a special type of queue where each element is assigned a priority value. Elements are processed according to priority rather than arrival order.

Standard Queue

Processes by arrival order — FIFO

Priority Queue

Processes by priority value — most urgent first



Why Priority Queue? — Hospital Example

In many real-world systems, some tasks are more important than others. A hospital emergency room is a classic example — patients are treated by urgency, not arrival time.

Arrival Order

Patient	Priority
Patient A	3 (Low)
Patient B	1 (High)
Patient C	2 (Medium)

Treatment Order

01

Patient B

Priority 1 — treated first

02

Patient C

Priority 2 — treated second

03

Patient A

Priority 3 — treated last

Priority Queue Using Python List

Elements are stored as tuples of (priority, item). After each insertion, the list is sorted so the highest-priority element (lowest number) is always at the front.

```
pq = []  
pq.append((3, 'Task A'))  
pq.append((1, 'Task B'))  
pq.append((2, 'Task C'))  
pq.sort()  
print(pq)
```

Output:

```
[(1, 'Task B'), (2, 'Task C'), (3, 'Task A')]
```

❏ Lower number = higher priority. Python's `sort()` on tuples sorts by the first element (priority) automatically.

Enqueue Method

```
def enqueue(self, priority,
item):
    self.queue.append(
        (priority, item)
    )
    self.queue.sort()
```

Inserts a new element and automatically re-sorts by priority.

Dequeue & Display

```
def dequeue(self):
    if len(self.queue) == 0:
        return "Queue Empty"
    return self.queue.pop(0)

def display(self):
    print(self.queue)
```

Removes and returns the highest-priority element. Display shows all current elements.

Complete Priority Queue Program

```
class PriorityQueue:
    def __init__(self):
        self.queue = []

    def enqueue(self, priority, item):
        self.queue.append((priority,
item))
        self.queue.sort()

    def dequeue(self):
        if len(self.queue) == 0:
            return "Queue Empty"
        return self.queue.pop(0)

    def display(self):
        print(self.queue)

pq = PriorityQueue()
pq.enqueue(3, "Task A")
pq.enqueue(1, "Task B")
pq.enqueue(2, "Task C")
pq.display()
print("Processed =",
pq.dequeue())
pq.display()
```

Output: [(1, 'Task B'), (2, 'Task C'), (3, 'Task A')] → Processed = (1, 'Task B') → [(2, 'Task C'), (3, 'Task A')]

Applications of Priority Queue



Operating Systems

CPU scheduling
and process
management



Networking

Packet routing and
traffic management



Healthcare

Emergency patient
scheduling by
urgency



Artificial Intelligence

A* search algorithm
and pathfinding



Cloud Computing

Resource
scheduling across
distributed
systems

Queue vs. Priority Queue

While both are linear data structures, Queue and Priority Queue differ fundamentally in how elements are ordered and processed.

Feature	Queue	Priority Queue
Principle	FIFO	Priority Based
Insertion	At Rear	Based on Priority
Deletion	From Front	Highest Priority First
Processing	Arrival Order	Priority Order
Example	Ticket Queue	CPU Scheduler

Time Complexity Comparison

Understanding time complexity helps choose the right structure. Queue operations are generally faster due to simpler ordering rules.

Operation	Queue	Priority Queue
Insert	$O(1)$	$O(n)$ — due to sort
Delete	$O(1)$	$O(1)$
Access Front	$O(1)$	$O(1)$
Display	$O(n)$	$O(n)$

- Priority Queue insertion is $O(n)$ because the list must be re-sorted after every enqueue operation.

✓ Queue Advantages

- Simple implementation
- Efficient $O(1)$ processing
- Fair scheduling — no element is skipped
- Maintains strict order of arrival
- Useful in real-time systems

✓ Priority Queue Advantages

- Processes important tasks first
- Efficient scheduling for critical systems
- Supports dynamic priorities
- Essential in operating systems
- Improves overall resource utilization

Choose a Queue when fairness and arrival order matter. Choose a Priority Queue when urgency and importance must override arrival order.

1

Queue Class Implementation

Implement a Queue using a Python class with operations: Enqueue, Dequeue, Front, Rear, Display.

2

Queue Operations Practice

Insert 10, 20, 30, 40 into a Queue. Then perform two dequeue operations and display the result.

3

Priority Queue Implementation

Implement a Priority Queue. Insert: Task A (5), Task B (2), Task C (1), Task D (4). Display the processing order.

Key Takeaways



Queue = FIFO

First In, First Out — insertion at rear, deletion at front

Priority Queue

Elements processed by priority value, not arrival order

Python Lists

Use `append()` and `pop(0)` for Queue; tuples + `sort()` for Priority Queue

Real-World Use

Queues power scheduling, networking, printing, and AI systems

- **Linear Data Structures**
 - Sequential organization of data
 - Introduction to Queue and Priority Queue concepts
- **Queue Fundamentals**
 - Definition of Queue
 - FIFO (First In, First Out) principle
 - Real-life examples of queue systems
- **Queue Operations**
 - Enqueue
 - Dequeue
 - Front
 - Rear
 - isEmpty
 - Display
- **Queue Implementation Using Python**
 - Queue representation
 - Queue implementation using Python lists
 - User-defined Queue class
- **Applications of Queue**
 - Ticket reservation systems
 - Printer queues
 - CPU scheduling
 - Network transmission systems
- **Priority Queue Fundamentals**
 - Definition and characteristics
 - Priority-based processing
 - Difference between Queue and Priority Queue
- **Priority Queue Implementation in Python**
 - Tuple-based storage
 - Sorting by priority
 - User-defined Priority Queue class
- **Applications of Priority Queue**
 - Operating systems
 - Networking
 - Healthcare systems
 - Artificial Intelligence
 - Cloud computing applications
- **Queue vs Priority Queue**
 - Processing mechanisms
 - Operational comparison
 - Appropriate use cases
- **Time Complexity and Advantages**
 - Complexity analysis
- Advantages and limitations of Queue and Priority Queue

Reference

Alvaro Fuentes, Become a Python Data Analyst – By Packt Publishing (2018)

Bharti Motwani, Data Analytics using Python – By Wiley (2020)

Jules S. Damji, Learning Spark: Lightning-Fast Data Analytics, Second Edition – By Shroff/O'Reilly (2020)

Data Science and Machine Learning using Python – 10 August 2022, MGH, 2022